

Exercise 1 Analysis: Comparison of RLS, (1+1)EA, and Genetic Algorithm

Introduction

This analysis compares the performance of three algorithms - RLS, (1+1)EA (OnePlusOneEA), and my Genetic Algorithm from Assignment 2 - across 11 different problem instances. Each algorithm was run 30 times on each instance with a budget of 10,000 function evaluations. The plots show the mean performance with standard deviation bands.

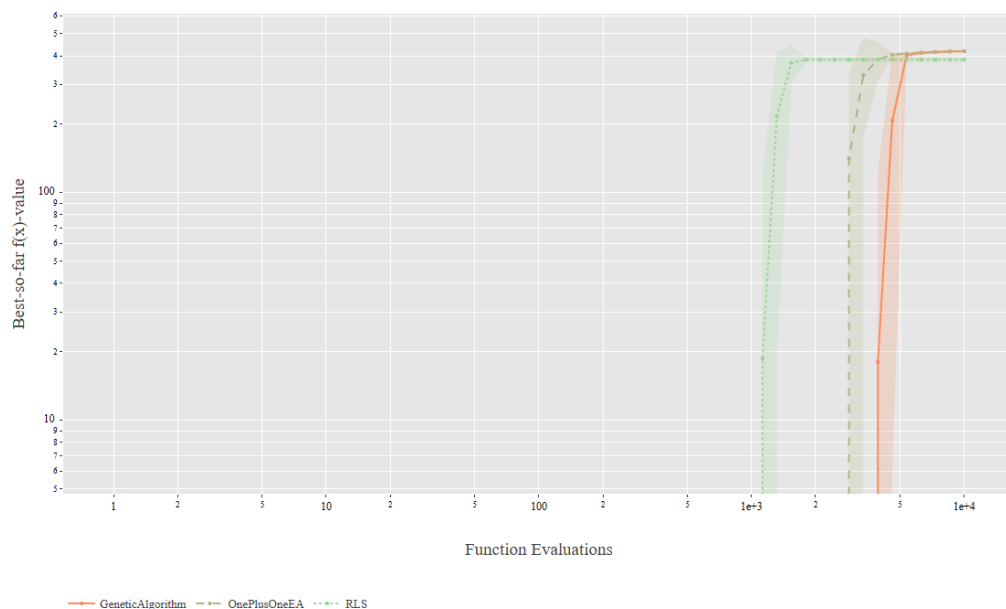
The instances include:

- MaxCoverage: 2100, 2101, 2102, 2103
- MaxInfluence: 2200, 2201, 2202, 2203
- PackWhileTravel: 2300, 2301, 2302

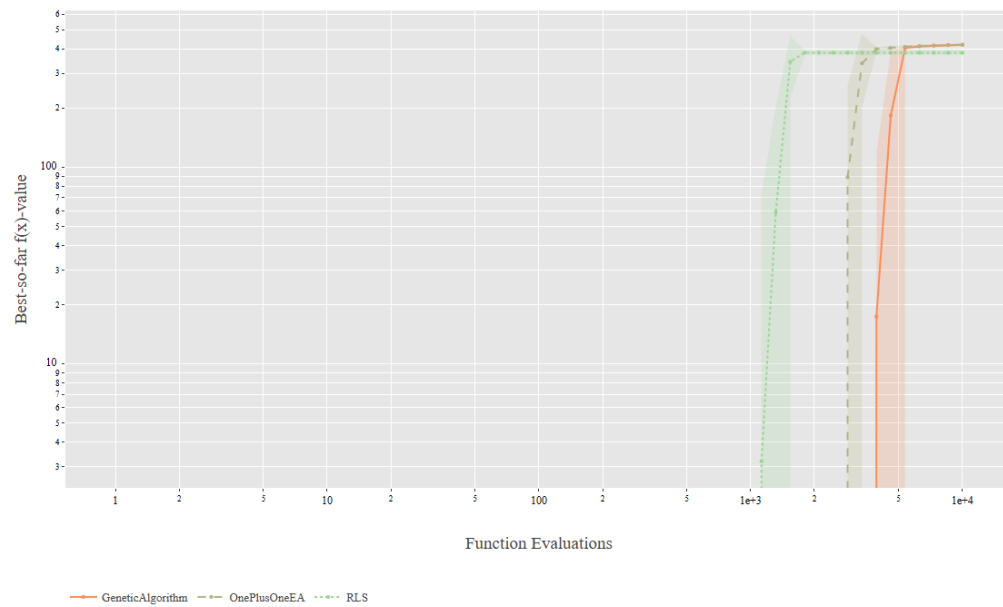
Results by Problem Type

MaxCoverage Problems (2100-2103)

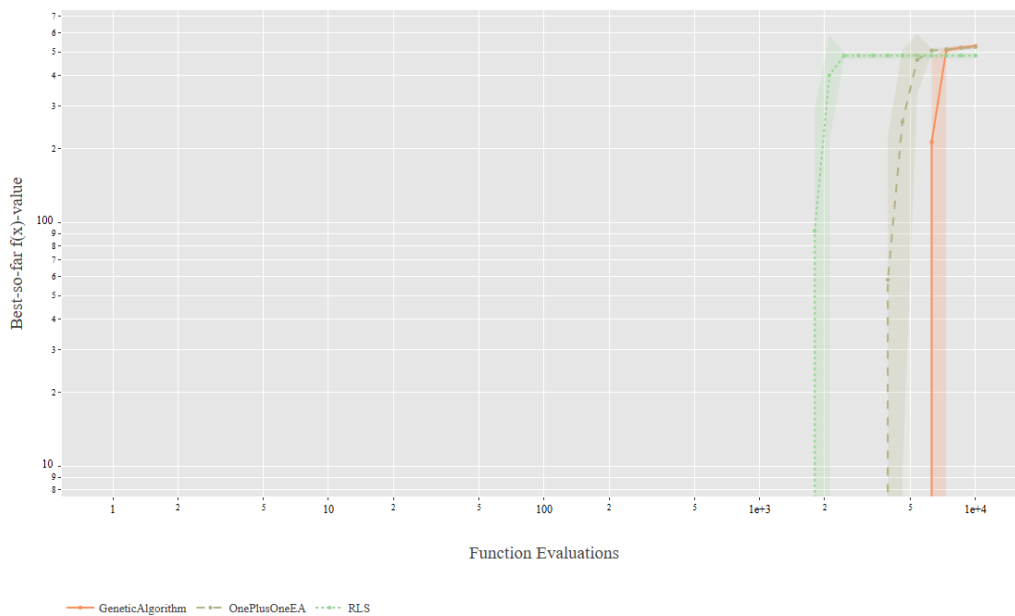
Looking at the four MaxCoverage instances, there's a consistent pattern across all of them:



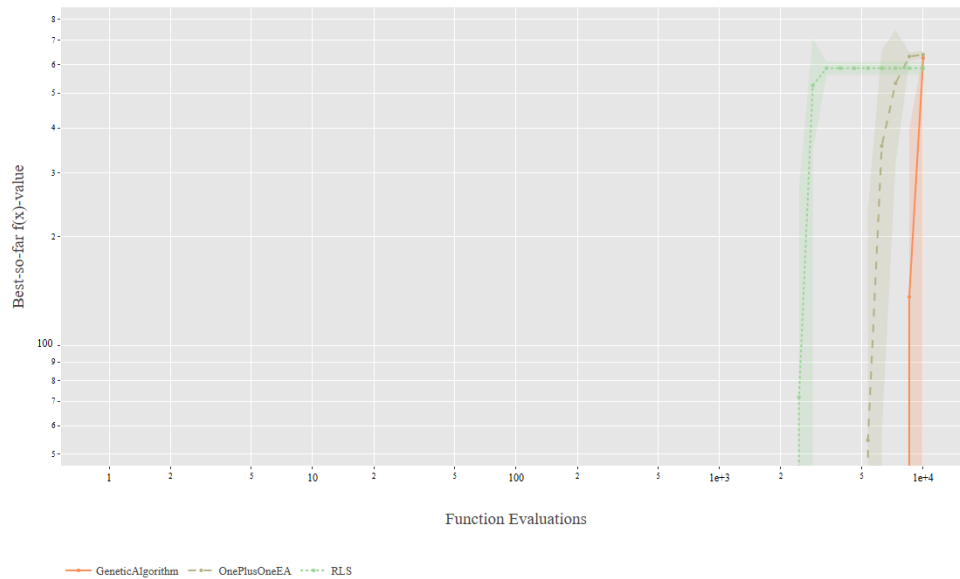
Instance 2100: RLS (green line) reaches around 4.0 fitness and converges fastest, getting there by about 1,500-2,000 evaluations. OnePlusOneEA (dotted line) performs very similarly to RLS, reaching the same fitness level but maybe slightly slower. My Genetic Algorithm (orange dashed line) reaches the same final fitness but takes more evaluations to get there - it's still improving even past 2,000 evaluations.



Instance 2101: Pretty much the same story as 2100. RLS converges quickest, OnePlusOneEA is close behind, and the GA is slower. All three reach around 4.0 fitness.



Instance 2102: Similar pattern but now the algorithms reach around 5.0 fitness. RLS still fastest, OnePlusOneEA similar, GA slower.

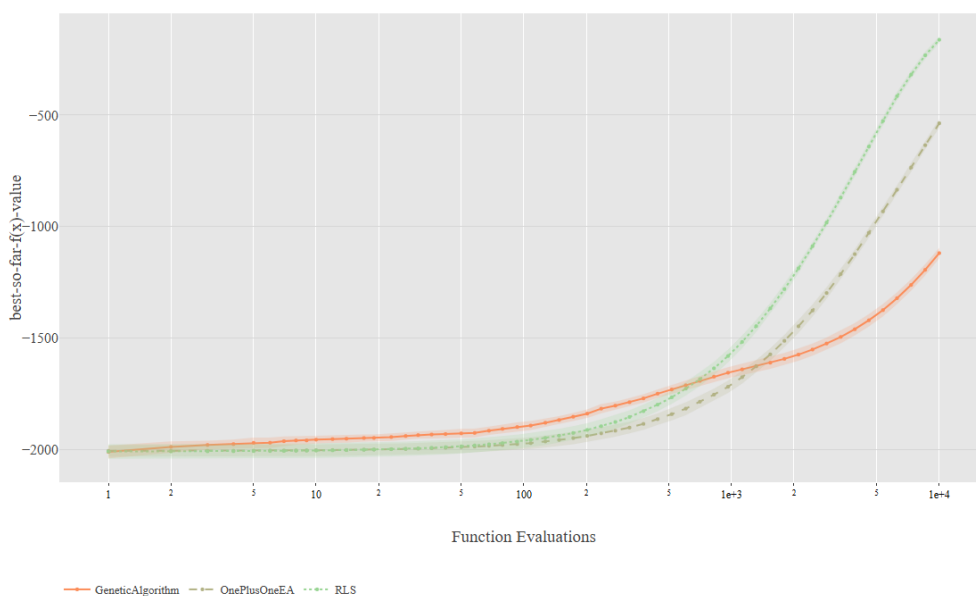


Instance 2103: All three algorithms reach about 6.0 fitness. RLS converges first (around 1,000-1,500 evals), then OnePlusOneEA, then the GA. By 10,000 evaluations they're all at roughly the same place, but RLS got there much faster.

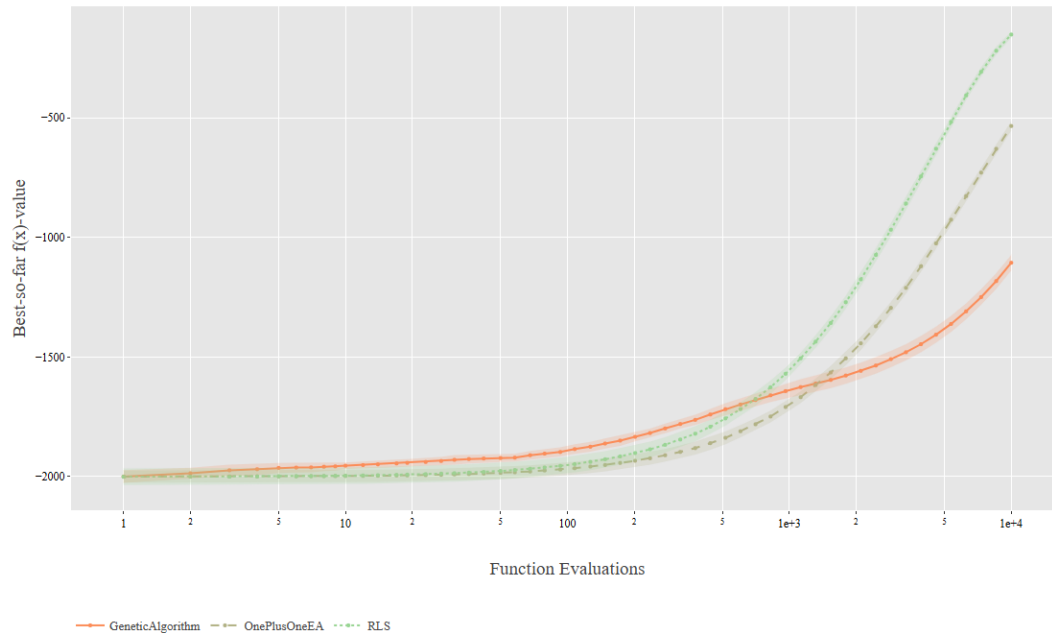
Overall for MaxCoverage: RLS is consistently the fastest to converge. OnePlusOneEA performs very similarly. My GA gets to similar final solutions but uses more evaluations. None of the algorithms seem to struggle with these problems - they all reach good solutions within the budget.

MaxInfluence Problems (2200-2203)

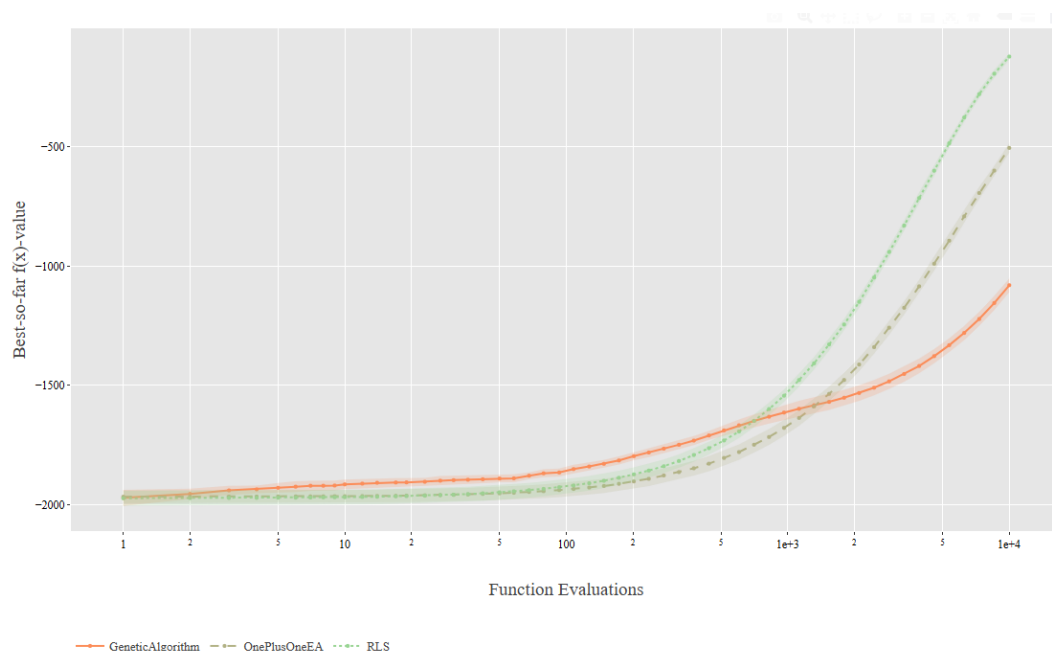
These problems look very different from MaxCoverage. The fitness values are negative (ranging from about -2000 to 0), and all three algorithms show much bigger performance differences.



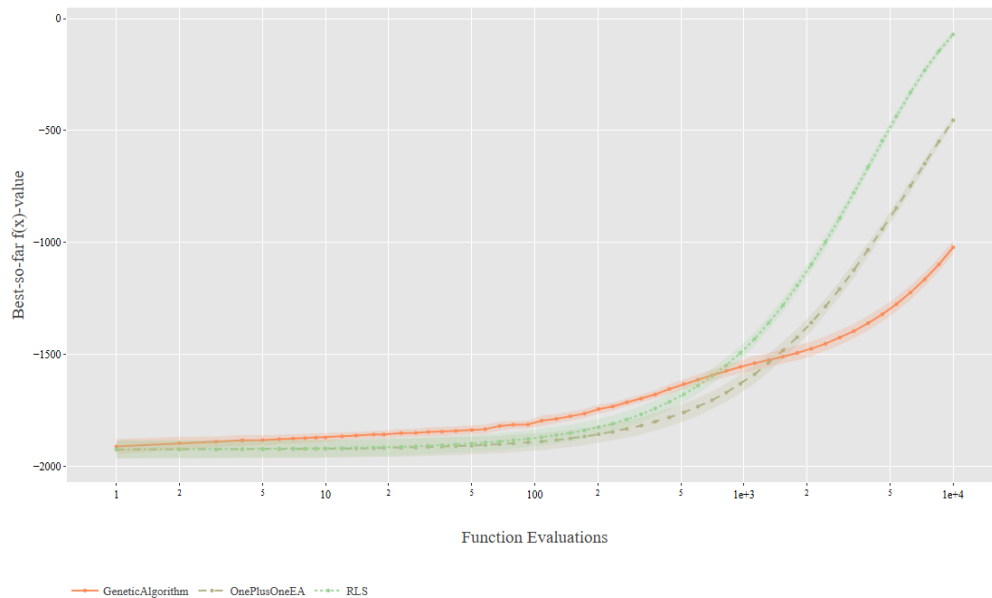
Instance 2200: RLS reaches around -200, OnePlusOneEA gets to about -500, and my GA only reaches around -1100. That's a gap of about 300 between RLS and OnePlusOneEA, and about 600 between OnePlusOneEA and the GA. None of them seem to have fully converged by 10,000 evaluations - they're all still improving.



Instance 2201: Very similar to 2200. RLS around -200, OnePlusOneEA around -500, GA around -1100. The curves look almost identical to 2200.



Instance 2202: Same pattern again. RLS $\sim$ -200, OnePlusOneEA $\sim$ -500, GA $\sim$ -1100. All three are still improving at the end of the budget.

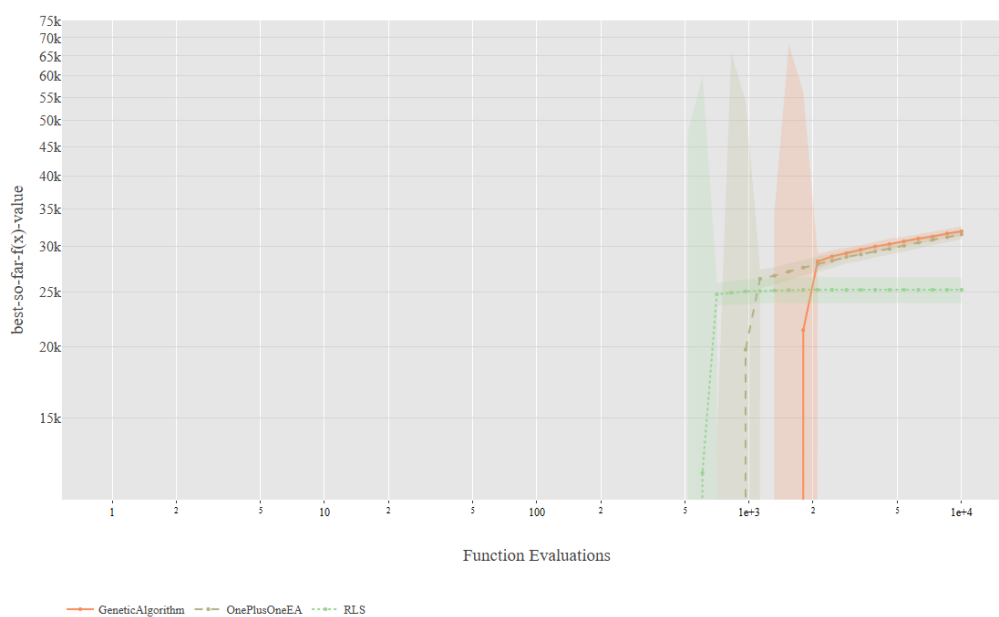


Instance 2203: Similar but slightly better fitness values. RLS reaches about -100 (better than the other MaxInfluence instances), OnePlusOneEA around -500, GA around -1050.

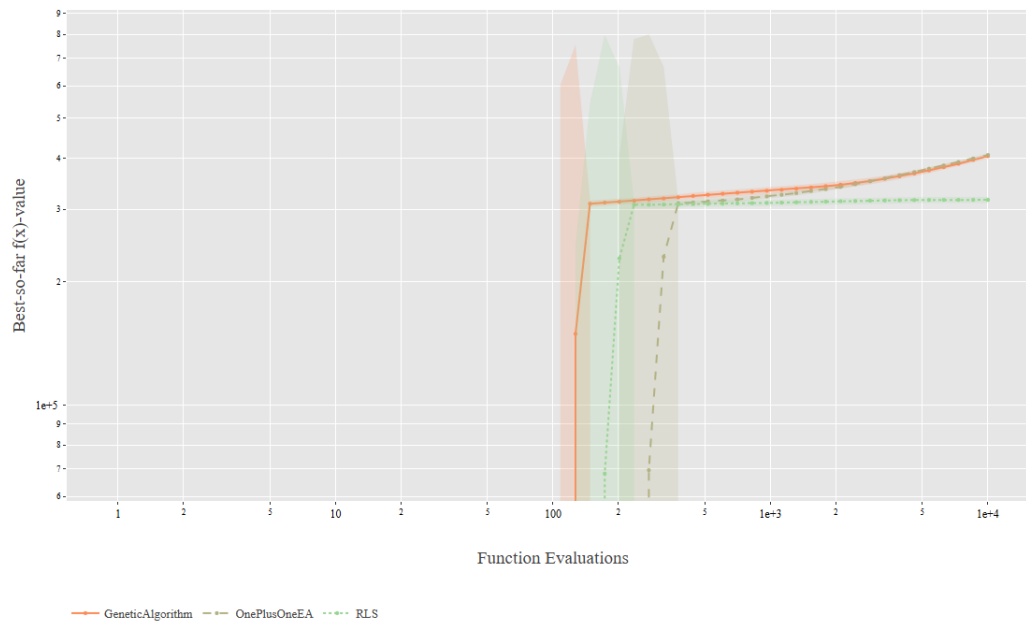
Overall for MaxInfluence: These are clearly harder problems. RLS performs significantly better than the other two algorithms - the difference is substantial (900 fitness units between RLS and GA). The negative fitness values suggest these might be minimisation problems or have some different structure. All algorithms are still improving when they hit the 10,000 evaluation limit, so a larger budget might help.

PackWhileTravel Problems (2300-2302)

These problems are completely different from the first two types. The fitness values are huge - in the hundreds of thousands to millions - and the plots use a log scale on the y-axis.

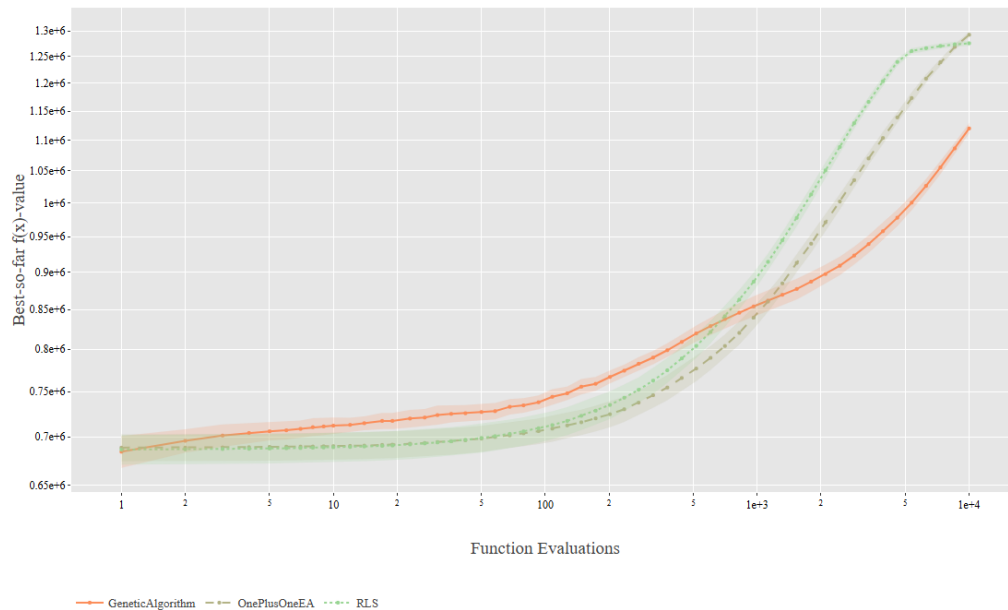


Instance 2300: The fitness values range from about 10,000 to 75,000. RLS and OnePlusOneEA behave pretty similarly here - they both make a sharp jump around 1,000 evaluations and then level off at roughly 25,000-27,000. The GA is different though. It jumps later, around 2,000 evaluations, but then keeps improving gradually instead of plateauing. By the end of the budget, the GA reaches about 31,000-32,000, which is actually higher than the other two algorithms. All three show these step-like improvements where they jump up and then flatten out.



Instance 2301: This instance is really unusual. The y-axis scale is strange, ranging from about 10^{-5} up to 9. The GA jumps very early - around evaluation 100 it shoots up to about 3.0, and then slowly climbs to around 3.8-4.0 by the end. The GA's confidence bands are really wide here, meaning the runs varied a lot from each other.

What's weird is what happens with RLS and OnePlusOneEA. Instead of improving like on other instances, they actually drop dramatically around evaluation 200-300. They fall from their starting values down to about 3.0-3.1 and just stay there for the rest of the budget. I haven't seen this dropping behavior on any of the other instances. All three algorithms plateau pretty early on this problem.



Instance 2302: The fitness values here are huge, ranging from about 0.65 million to 1.3 million. RLS shows the steepest improvement - it starts around 0.69 million and climbs to about 1.27 million, with most of that improvement happening between 1,000 and 5,000 evaluations. OnePlusOneEA follows a very similar path and ends up at about 1.26 million.

The GA takes a different approach. It starts at a similar level (around 0.68 million) but improves more gradually throughout the entire run and therefore reaching about 1.12 million by the end. So RLS and OnePlusOneEA do achieve higher final fitness values, but the GA keeps improving steadily the whole time without plateauing like the others do. This is actually the first PackWhileTravel instance where all three algorithms show relatively smooth convergence curves instead of those dramatic jumps we saw in the other instances.

Overall for PackWhileTravel: These problems have very different characteristics with large fitness scales and distinct improvement patterns. The discrete jumps and plateau behaviors suggest they might involve integer constraints or packing decisions. Performance is much more instance-dependent here: on instance 2300, the GA actually achieves the highest final fitness ($\approx 32,000$) compared to RLS and OnePlusOneEA ($\approx 25,000$); on instance 2301, all algorithms show unusual behavior with RLS and OnePlusOneEA experiencing sudden drops; on instance 2302, RLS and OnePlusOneEA reach higher final fitness ($\approx 1.27\text{M}$ and 1.26M) than the GA ($\approx 1.12\text{M}$), though the GA shows consistent improvement throughout.

Algorithm Comparison

RLS

- Best overall performance across most instances (clearly best on all MaxInfluence instances)
- Fastest convergence on most MaxCoverage instances
- Tighter confidence bands (more consistent across runs)
- Achieves highest fitness on PackWhileTravel instance 2302 ($\approx 1.27\text{M}$), but reaches lower fitness than GA on instance 2300 ($\approx 25\text{k}$ vs $\approx 32\text{k}$)

OnePlusOneEA

- Usually the second best
- Very similar to RLS on MaxCoverage problems
- Middle performance on MaxInfluence (better than GA, worse than RLS)
- Moderate performance on PackWhileTravel

Genetic Algorithm

- Slowest convergence on MaxCoverage instances but reaches a similar final fitness
- Significantly worse on MaxInfluence instances (~900 fitness units behind RLS)
- Mixed results on PackWhileTravel: achieves highest fitness on instance 2300 (~32k), shows high variability on 2301, but reaches lower fitness than RLS/OnePlusOneEA on 2302 (~1.12M vs ~1.27M)
- Generally wider confidence bands (more variability between the runs)

Discussion

The results show that algorithm performance really depends on the problem type. RLS works best overall, especially on MaxCoverage and MaxInfluence problems. My GA doesn't perform as well on most instances, which is interesting because GAs are supposed to be good general-purpose optimisers.

The MaxInfluence problems are clearly the hardest - all algorithms are still improving at 10,000 evaluations, and the performance gaps between algorithms are largest here. It looks like these problems might benefit from a larger evaluation budget.

The PackWhileTravel problems are unusual with their massive fitness scales. Interestingly, the GA achieves the highest fitness on instance 2300 which therefore suggests that the population-based approach can be advantageous on certain problem structures. However, on instance 2302, RLS and OnePlusOneEA outperform the GA which therefore indicates that the relationship between algorithm choice and problem structure is complex.

One thing I notice is that simpler algorithms (RLS, OnePlusOneEA) generally work better than my more complex GA. This matches what we learned about submodular problems that greedy-like approaches often work well for these problems.

Conclusion

RLS is the best algorithm for most of these instances. It has a faster convergence and higher quality solutions particularly on MaxCoverage problems and MaxInfluence problems. OnePlusOneEA is a secure second alternative that has the same performance though slightly lower. My GA underperforms on most instances but shows it can be competitive on certain problem structures (like PackWhileTravel 2300).

The major lesson is that the choice of algorithm is important and varies according to the type of problem. In the submodular optimisation problems that are tested in this case, simpler

algorithms such as RLS appear better than more complicated population-based algorithms such as GAs.